
```

%Tableros de Butcher para m'etodos adaptativos
function [A, b, bhat, c, s, orden] = CoefsRKAd(metodo)

switch upper(metodo)

    %M'ETODOS EXPL'ICITOS:
    case 'RK2'
        A      = [ 0.0      0.0;
                   0.5      0.0 ];
        b      = [ 0.0;      1.0 ];
        bhat   = [ 1.0;      0.0 ];
        c      = [ 0.0;      0.5 ];
        s      = 2;
        orden  = 2;

    %M'ETODOS IMPL'ICITOS:
    case 'RK3I' %Ascher et al ANM1997
        A      = [ 0.0      0.0      0.0      0.0      0.0
                   0.0      1/2      0.0      0.0      0.0
                   0.0      1/6      1/2      0.0      0.0
                   0.0      -1/2     1/2      1/2      0.0
                   0.0      3/2      -3/2     1/2      1/2 ];
        b      = [ 0.0      3/2      -3/2     1/2      1/2 ].';
        bhat   = [ 0.0      1.0      0.0      0.0      0.0 ].';
        c      = sum(A, 2);
        s      = size(A, 1);
        orden  = 3;

    case 'RK4I' %Kennedy and Carpenter, ANM2019

        [A, bhat, c] = ESDIRK4_3_7L_2_SA();
        b            = A(end, :).';
        s            = size(A, 1);
        orden        = 4;

    otherwise
        error('Metodo desconocido')
end

end

function [Am, bhat, cv] = ESDIRK4_3_7L_2_SA()

    Am      = zeros(7, 7);

    for ii=2:7
        Am(ii, ii) = 1/8;
    end
    Am(3, 2) = -39188347878/1513744654945;
    Am(4, 2) = 1748874742213/5168247530883;
    Am(4, 3) = -1748874742213/5795261096931;
    Am(5, 2) = -6429340993097/17896796106705;

```

```
Am(5,3)    = 9711656375562/10370074603625;  
Am(5,4)    = 1137589605079/3216875020685;  
Am(6,2)    = 405169606099/1734380148729;  
Am(6,3)    = -264468840649/6105657584947;  
Am(6,4)    = 118647369377/6233854714037;  
Am(6,5)    = 683008737625/4934655825458;  
Am(7,2)    = -5649241495537/14093099002237;  
Am(7,3)    = 5718691255176/6089204655961;  
Am(7,4)    = 2199600963556/4241893152925;  
Am(7,5)    = 8860614275765/11425531467341;  
Am(7,6)    = -3696041814078/664156663007;
```

```
cv          = zeros(7,1);  
cv(2)       = 1/4;  
cv(3)       = 1200237871921/16391473681546;  
cv(4)       = 1/2;  
cv(5)       = 395/567;  
cv(6)       = 89/126;  
cv(7)       = 1.0;
```

```
for ii=2:7  
    Am(ii,1) = cv(ii)-sum(Am(ii,2:ii));  
end
```

```
bhat        = zeros(7,1);  
bhat(2)     = -1517409284625/6267517876163;  
bhat(3)     = 8291371032348/12587291883523;  
bhat(4)     = 5328310281212/10646448185159;  
bhat(5)     = 5405006853541/7104492075037;  
bhat(6)     = -4254786582061/7445269677723;  
bhat(7)     = 19/140;  
bhat(1)     = 1.0 - sum(bhat(2:7));
```

```
end
```

Published with MATLAB® R2025a

```

function [A, b, c, s] = CoefsRK(metodo)

switch upper(metodo)

%M'ETODOS EXPL'ICITOS:
case 'RK1'
    A = [ 0.0 ];
    b = [ 1.0 ];
    c = [ 0.0 ];
    s = 1;
case 'RK2'
    A = [ 0.0      0.0;
          0.5      0.0 ];
    b = [ 0.0; 1.0 ];
    c = [ 0.0; 0.5 ];
    s = 2;
case 'RK4' %RK4 cl'asico
    A = [ 0.0      0.0      0.0      0.0;
          0.5      0.0      0.0      0.0;
          0.0      0.5      0.0      0.0;
          0.0      0.0      1.0      0.0 ];
    b = [ 1/6; 1/3; 1/3; 1/6 ];
    c = [ 0.0; 0.5; 0.5; 1.0 ];
    s = 4;

%M'ETODOS IMPL'ICITOS:
case 'RK1I'
    A = [ 0.0      0.0;
          0.0      1.0 ];
    b = [ 0.0; 1.0 ];
    c = [ 0.0; 1.0 ];
    s = 2;
case 'RK2I'
    A = [ 0.0      0.0;
          0.0      0.5 ];
    b = [ 0.0; 1.0 ];
    c = [ 0.0; 0.5 ];
    s = 2;

case 'RK3I' %Ascher et al., ANM1997
    A = [ 0.0      0.0      0.0      0.0      0.0
          0.0      1/2      0.0      0.0      0.0
          0.0      1/6      1/2      0.0      0.0
          0.0      -1/2     1/2      1/2      0.0
          0.0      3/2      -3/2     1/2      1/2 ];
    b = [ 0.0      3/2      -3/2     1/2      1/2 ].';
    c = sum(A, 2);
    s = size(A, 1);

case 'RK4I' %Kennedy and Carpenter, ANM2019

    [A, bhat, c] = ESDIRK4_3_7L_2_SA();

```

```

        b          = A(end,:).';
        s          = size(A,1);

        otherwise
            error('Metodo desconocido')
        end

end

function [Am, bhat, cv] = ESDIRK4_3_7L_2_SA()

    Am          = zeros(7,7);

    for ii=2:7
        Am(ii,ii) = 1/8;
    end
    Am(3,2)      = -39188347878/1513744654945;
    Am(4,2)      = 1748874742213/5168247530883;
    Am(4,3)      = -1748874742213/5795261096931;
    Am(5,2)      = -6429340993097/17896796106705;
    Am(5,3)      = 9711656375562/10370074603625;
    Am(5,4)      = 1137589605079/3216875020685;
    Am(6,2)      = 405169606099/1734380148729;
    Am(6,3)      = -264468840649/6105657584947;
    Am(6,4)      = 118647369377/6233854714037;
    Am(6,5)      = 683008737625/4934655825458;
    Am(7,2)      = -5649241495537/14093099002237;
    Am(7,3)      = 5718691255176/6089204655961;
    Am(7,4)      = 2199600963556/4241893152925;
    Am(7,5)      = 8860614275765/11425531467341;
    Am(7,6)      = -3696041814078/6641566663007;

    cv          = zeros(7,1);
    cv(2)       = 1/4;
    cv(3)       = 1200237871921/16391473681546;
    cv(4)       = 1/2;
    cv(5)       = 395/567;
    cv(6)       = 89/126;
    cv(7)       = 1.0;

    for ii=2:7
        Am(ii,1) = cv(ii)-sum(Am(ii,2:ii));
    end

    bhat        = zeros(7,1);
    bhat(2)     = -1517409284625/6267517876163;
    bhat(3)     = 8291371032348/12587291883523;
    bhat(4)     = 5328310281212/10646448185159;
    bhat(5)     = 5405006853541/7104492075037;
    bhat(6)     = -4254786582061/7445269677723;
    bhat(7)     = 19/140;
    bhat(1)     = 1.0 - sum(bhat(2:7));

end

```

Published with MATLAB® R2025a

```

function ED0s_Blasius()

%-----
%DATOS Y FUNCIONES AUXILIARES:

%Corriente incidente:
U          = 1.2;
nu         = 0.01;

%Funci'on con la ecuaci'on diferencial. Nota: definimos F=[f, f', f'']:
function dF_deta = BlasiusFun(eta, F)
    dF_deta     = [ F(2); F(3); -0.5*F(1)*F(3) ];
end

%Funci'on que, recibida f''(0), devuelve la soluci'on del ED0,
%integrada como se pide en el enunciado:
function [etav, Fm] = BlasiusRK(fpp0)
    F0         = [ 0.0; 0.0; fpp0 ];
    [etav, Fm] = RungeKuttaE(@BlasiusFun, 0.0, F0, 'RK4', 1e-2, 10.0);
end

%-----
%RESOLUCI'ON POR ANDERSON:

%Funci'on residuo sin preconditionar:
function r = rfun(fpp0)
    [~, Fm]     = BlasiusRK(fpp0);
    r           = Fm(2,end)-1.0;
end

%Representamos r(f0'') en el intervalo (0, 0.8). NO SE PIDE EN EL
ENUNCIADO,
%pero se incluye aqu' a modo ilustrativo para justificar la aproximaci'on
%  $dr/df_0'' \sim (r(0.8)-r(0))/0.8$ 
figure()
f0ppv = linspace(0, 0.8, 20);
rv     = zeros(size(f0ppv));
for ii=1:length(f0ppv)
    rv(ii) = rfun(f0ppv(ii));
end
plot(f0ppv, rv)
xlabel('f_0''''')
ylabel('r')
drawnow %para que matlab dibuje las gr'aficas antes de seguir

%Definimos residuo preconditionado:
dr_dfpp0 = (rfun(0.8)-rfun(0.0))/0.8;
function g = gfun(fpp0)
    g       = rfun(fpp0)/dr_dfpp0;
end

%Llamamos al m'etodo de Anderson. S'olo se puede imponer una condici'on

```

```

%secante porque s'olo hay una inc'ognita:
[fpp0, ~, flag] = Anderson1(@gfun, 0.8, 1e-10, 100, 1);
if flag<0
    warning('Anderson no ha convergido')
end

%-----
%REPRESENTACION DE LA SOLUCION:

%Hallamos la soluci'on para el valor de fpp0 encontrado:
[etav, Fm] = BlasiusRK(fpp0);

%Animaci'on con la evoluci'on de la capa l'imate:
nPlot = 100;
xPlotv = linspace(0.01, 1.5, nPlot);
figure()
for ii=1:nPlot
    %Valor de x:
    x = xPlotv(ii);
    %Valores de y = eta/sqrt(U/(nu*x))
    yv = etav/sqrt(U/(nu*x));
    %Valores de u = U * f'
    uv = U*Fm(2, :);
    %Dibujo:
    plot(uv, yv, 'b')
    xlabel('u')
    ylabel('y')
    title(sprintf('x=%.2f', x))
    ylim([0,1.4])
    pause(0.1)
end
end

```

Published with MATLAB® R2025a

```

%Crecimiento de una llama en una cerilla.
function EDOs_cerilla_RKAdI(delta, metodo, Tol)

%Datos:
y0      = delta;
tf      = 2/delta;

%Ecuaci'on a integrar:
function [ydot, dydot_dy] = ydot_fun(t, y, CalcJ)

    ydot    = y^2 - y^3;
    if CalcJ
        dydot_dy    = 2*y - 3*y^2;
    else
        dydot_dy    = NaN;
    end

end

%Exact solution:
function y = ytheorfun(t)
    a      = 1/delta -1 ;
    y      = 1.0./(1.0+lambertw(a*exp(a-t)));
end

%Soluci'on num'ERICA:
tic
[tv, ym]  = RungeKuttaAdI(@ydot_fun, 0.0, [y0], metodo, Tol, 1e-6, tf);
toc

%Representamos soluci'on:
ym_ex     = ytheorfun(tv);
figure()
plot(tv, ym(1,:), 'x-b')
hold on
plot(tv, ym_ex(1,:), 'r')
legend('Numerica', 'Exacta', 'location', 'bestoutside')

%Calculamos error:
ym_ex     = ytheorfun(tv);
err       = max(abs(ym_ex(1,:)-ym(1,:)));
display(err)

%Deltat medio:
Deltat_medio = tf/(length(tv)-1);
display(Deltat_medio)

end

```

```
%Crecimiento de una llama en una cerilla.
function EDOs_cerilla_RKI(delta, metodo, Deltat)

%Datos:
y0      = delta;
tf      = 2/delta;

%Ecuaci'on a integrar:
function [ydot, dydot_dy] = ydot_fun(t, y, CalcJ)

    ydot    = y^2 - y^3;
    if CalcJ
        dydot_dy = 2*y - 3*y^2;
    else
        dydot_dy = NaN;
    end

end

%Exact solution:
function y = ytheorfun(t)
    a      = 1/delta - 1 ;
    y      = 1.0./(1.0+lambertw(a*exp(a-t)));
end

%Soluci'on num'ERICA:
tic
[tv, ym]  = RungeKuttaI(@ydot_fun, 0.0, [y0], metodo, Deltat, tf);
toc

%Representamos soluci'on:
ym_ex     = ytheorfun(tv);
figure()
plot(tv, ym(1,:), 'xb')
hold on
plot(tv, ym_ex(1,:), 'r')
legend('Numerica', 'Exacta', 'location', 'bestoutside')

%Calculamos error:
ym_ex     = ytheorfun(tv);
err       = max(abs(ym_ex(1,:)-ym(1,:)));
display(err)

end
```

Published with MATLAB® R2025a

```

%Masa-muelle-amortiguador:
function EDOs_masa_muelle(metodo, Deltat, tf)
    %Deltat=0.01, tf=20.0 como primer ejemplo.
    %Luego dividir Deltat entre 10 y ver cómo el error disminuye.

    %Datos:
    m          = 1.0;
    c          = 0.1;
    k          = 2.0;
    x0         = 1.0;
    v0         = -1.5;

    %Ecuación a integrar:
    function ydot = ydot_fun(t,y)

        x      = y(1);
        v      = y(2);
        ydot    = [ v; -k*x/m-c*v/m ]; %ydot=f en los apuntes

    end

    %Solución exacta:
    function y = ytheor_fun(t) %t puede ser un vector fila

        lambda1 = (-c + sqrt(c^2 - 4*m*k))/(2*m);
        lambda2 = (-c - sqrt(c^2 - 4*m*k))/(2*m);
        Cv      = [ 1, 1; lambda1, lambda2 ] \ [ x0; v0 ];
        x       = real(Cv(1)*exp(lambda1*t) + Cv(2)*exp(lambda2*t));
        v       = real(Cv(1)*lambda1*exp(lambda1*t) +
        Cv(2)*lambda2*exp(lambda2*t));
        y       = cat(1, x, v);

    end

    %Solución numérica:
    [tv, ym] = RungeKuttaE(@ydot_fun, 0.0, [x0; v0], metodo, Deltat, tf);

    %Representamos solución:
    ym_ex = ytheor_fun(tv);
    figure()
    plot(tv, ym_ex(1,:), '--b')
    hold on
    plot(tv, ym_ex(2,:), '--g')
    plot(tv, ym(1,:), 'b')
    plot(tv, ym(2,:), 'g')
    xlabel('t')
    legend('x_{ex}', 'v_{ex}', 'x_{num}', 'v_{num}', 'location',
    'bestoutside')

    %Calculamos errores:
    err_x = max(abs(ym_ex(1,:)-ym(1,:)));
    err_v = max(abs(ym_ex(2,:)-ym(2,:)));

```

```
display(err_x)
display(err_v)
```

```
end
```

Published with MATLAB® R2025a

```
%Prothero--Robinson:
% Probar primero con mu=-1, RK4, Deltat=1e-2, tf=20.
% Probar luego con mu=-1000 y mu=-10^6
function ED0s_ProtheroRobinson_RKE(mu, metodo, Deltat, tf)

    %Ecuaci'on a integrar:
    function ydot = ydot_fun(t,y)

        ydot    = [ mu*(y(1)-sin(t)) + cos(t) ]; %ydot = f en los apuntes

    end

    %Soluci'on num'ERICA:
    tic
    t0          = 0.0;
    y0          = [0.0];
    [tv, ym]    = RungeKuttaE(@ydot_fun, t0, y0, metodo, Deltat, tf);
    toc

    %Soluci'on exacta
    function y = ytheor_fun(t) %t puede ser un vector fila

        y        = sin(t);

    end

    %Representamos soluci'on:
    ym_ex        = ytheor_fun(tv);
    figure()
    plot(tv, ym_ex(1,:), 'b')
    hold on
    plot(tv, ym(1,:), 'c')

    %Calculamos error:
    err          = max(abs(ym_ex(1,:)-ym(1,:)));
    display(err)

end
```

Published with MATLAB® R2025a

```
%Prothero--Robinson:
% Probar primero con mu=-1, RK4, Deltat=1e-2, tf=20.
% Probar luego con mu=-1000 y mu=-10^6
%OJO! Llamar a m'etodo impl'icito (terminado en 'I')
function EDOs_ProtheroRobinson_RKI(mu, metodo, Deltat, tf)

    %Ecuaci'on a integrar:
    function [f, df_dy] = ydot_fun(t, y, CalcJ)

        f          = [ mu*(y(1)-sin(t)) + cos(t) ];
        if CalcJ
            df_dy    = [ mu ];
        else
            df_dy    = NaN;
        end

    end

    %Soluci'on num'ERICA:
    tic
    t0          = 0.0;
    y0          = [0.0];
    [tv, ym]    = RungeKuttaI(@ydot_fun, t0, y0, metodo, Deltat, tf);
    toc

    %Soluci'on exacta
    function y = ytheor_fun(t) %t puede ser un vector fila

        y          = sin(t);

    end

    %Representamos soluci'on:
    ym_ex       = ytheor_fun(tv);
    figure()
    plot(tv, ym_ex(1,:), 'b')
    hold on
    plot(tv, ym(1,:), 'c')

    %Calculamos error:
    err         = max(abs(ym_ex(1,:)-ym(1,:)));
    display(err)

end
```

Published with MATLAB® R2025a

```

%Probar con RK4I, tf = 100
function ED0s_Robertson_RKAdI(metodo, Tol, tf)

    %Ecuaci'on a integrar:
    function [ydot, dydot_dy] = ydot_fun(t,y,CalcJ)

        ydot          = [ -0.04*y(1)  + 1e4*y(2)*y(3);
                           +0.04*y(1)  - 1e4*y(2)*y(3)   - 3e7*y(2)*y(2);
                                                           + 3e7*y(2)*y(2) ];

        if CalcJ
            dydot_dy    = [ -0.04          +1e4*y(3)          +1e4*y(2);
                           +0.04          -1e4*y(3)-2*3e7*y(2)  -1e4*y(2);
                           0.0           2*3e7*y(2)
            ];
        else
            dydot_dy     = NaN;
        end

    end

    %Soluci'on num'ERICA:
    tic
    [tv, ym]            = RungeKuttaAdI(@ydot_fun, 0.0, [1; 0; 0], metodo, Tol,
1e-6, tf);
    toc

    Deltat_medio       = tf/(length(tv)-1);
    display(Deltat_medio)

    %Representamos soluci'on:
    figure()
    semilogx(tv, ym(1,:), '-xb')
    title('y_1')

    figure()
    semilogx(tv, ym(2,:), '-xg')
    title('y_2')

    figure()
    semilogx(tv, ym(3,:), '-xr')
    title('y_3')

end

```

Published with MATLAB® R2025a

```
%Probar con RK4, tf = 100
function EDOs_Robertson_RKE(metodo, Deltat, tf)

    %Ecuaci'on a integrar:
    function ydot = ydot_fun(t,y)

        ydot    = [ -0.04*y(1)  + 1e4*y(2)*y(3);
                    +0.04*y(1)  - 1e4*y(2)*y(3)    - 3e7*y(2)*y(2);
                                                         + 3e7*y(2)*y(2) ];

    end

    %Soluci'on num'ERICA:
    tic
    [tv, ym]    = RungeKuttaE(@ydot_fun, 0.0, [1; 0; 0], metodo, Deltat, tf);
    toc

    %Representamos soluci'on:
    figure()
    semilogx(tv, ym(1,:), '-xb')
    title('y_1')

    figure()
    semilogx(tv, ym(2,:), '-xg')
    title('y_2')

    figure()
    semilogx(tv, ym(3,:), '-xr')
    title('y_3')

end
```

Published with MATLAB® R2025a

```

%Probar con RK4I, tf = 100
function EDOs_Robertson_RKI(metodo, Deltat, tf)

    %Ecuaci'on a integrar:
    function [ydot, dydot_dy] = ydot_fun(t,y,CalcJ)

        ydot          = [ -0.04*y(1)  + 1e4*y(2)*y(3);
                           +0.04*y(1)  - 1e4*y(2)*y(3)   - 3e7*y(2)*y(2);
                                                           + 3e7*y(2)*y(2) ];

        if CalcJ
            dydot_dy    = [ -0.04          +1e4*y(3)          +1e4*y(2);
                           +0.04          -1e4*y(3)-2*3e7*y(2)  -1e4*y(2);
                           0.0           2*3e7*y(2)
            ];
        else
            dydot_dy     = NaN;
        end

    end

    %Soluci'on num'ERICA:
    tic
    [tv, ym]    = RungeKuttaI(@ydot_fun, 0.0, [1; 0; 0], metodo, Deltat, tf);
    toc

    %Representamos soluci'on:
    figure()
    semilogx(tv, ym(1,:), '-xb')
    title('y_1')

    figure()
    semilogx(tv, ym(2,:), '-xg')
    title('y_2')

    figure()
    semilogx(tv, ym(3,:), '-xr')
    title('y_3')

end

```

Published with MATLAB® R2025a

```

function RegEstabRungeKutta1(metodo, ax, bx, Nx, ay, by, Ny)

    %Coeficientes del m'etodo:
    [A,b,c,s] = CoefsRK(metodo);

    %Mallamos ejes real e imaginario:
    xv = linspace(ax, bx, Nx);
    yv = linspace(ay, by, Ny);

    %Funci'on de estabilidad. Nota: z=lambda*Deltat
    Xm = zeros(Nx,Ny); %Matriz con los valores de Re(z)
    Ym = zeros(Nx,Ny); %Matriz con los valores de Im(z)
    Rm = zeros(Nx,Ny); %Matriz con los valores de R(z)
    unos = ones(s,1);
    Im = eye(s);
    for ii=1:Nx
        for jj=1:Ny
            Xm(ii,jj) = xv(ii);
            Ym(ii,jj) = yv(jj);
            z = xv(ii)+1i*yv(jj);
            aux = (Im-z*A)\unos;
            Rm(ii,jj) = abs(1+z*(b.'*aux));
        end
    end

    %Pintamos región G<=1:
    figure()
    contourf(Xm, Ym, 1./Rm, [1.0, Inf])
    %Nota: si hacemos contourf(Xm, Ym, Rm, [1.0 Inf]), Matlab pinta en
    %blanco la región para la que |R|<1 (región estable) y en azul la región
    %para la que |R|>1 (región inestable). Para que sea al revés,
    %representamos 1/Rm en lugar de Rm.
    colormap('winter')
    grid on
    axis equal
    xlabel('Re(\lambda \Delta t)')
    ylabel('Im(\lambda \Delta t)')
    title(metodo, 'FontWeight', 'normal')

end

```

Published with MATLAB® R2025a

```

%Devuelve vector fila tv=[t_0, t_1, ..., t_n] y matriz
% ym = [y_0, y_1, ..., y_n], donde y_i = y(t_i) es un vector columna:
function [tv, ym] = RungeKuttaAdI(fun, t0, y0, metodo, Tol, Deltat0, tf)

    %Condici'on inicial:
    tn      = t0;
    yn      = y0(:);
    tv      = [tn];
    ym      = [yn];
    Deltatn = Deltat0;

    %Cargamos coeficientes:
    [A,b,bhat,...
     c,s,orden] = CoefsRKAd(metodo);

    %Matriz para guardar derivadas:
    fm        = zeros(length(yn), s);

    %Bucle hasta llegar a tf:
    while tn<tf

        %Derivada en t^n:
        fm(:,1) = fun(tn, yn, false);

        %Calcular soluci'on en t^(n+1) hasta que converja:
        while true

            %Paso de tiempo:
            tnp1 = tn + Deltatn;

            %Recortar paso de tiempo si es necesario:
            if tnp1>tf
                tnp1 = tf;
            end
            Deltatn = tnp1-tn;

            %Recorremos etapas:
            for ii=2:s
                %Variables auxiliares:
                t_ii = tn + c(ii)*Deltatn;
                S     = 0.0;
                for jj=1:ii-1
                    S = S + fm(:,jj)*A(ii,jj);
                end
                aux_ii = yn + Deltatn*S;
                %Resolver:
                [y_ii, ~, flag] = NewtonRaphson1(@ResiduoFun, yn,
0.01*Tol, 100);
                %Salimos del bucle for si no hay convergencia:
                if flag<0
                    break
                end
            end
        end
    end

```

```

        %Guardar derivada:
        fm(:,ii) = fun(t_ii, y_ii, false);
    end

    %Soluci'on en t^(n+1):
    S = 0.0;
    for jj=1:s
        S = S + fm(:,jj)*b(jj);
    end
    ynp1 = yn + Deltatn*S;

    %Estimaci'on del error:
    en = 0.0;
    for jj=1:s
        en = en + fm(:,jj)*(b(jj)-bhat(jj));
    end
    en = en*Deltatn;
    en = norm(en, Inf);

    %Ajustar paso de tiempo:
    if flag<0
        %Reducimos paso de tiempo:
        Deltatn = 0.5*Deltatn;
    elseif en>Tol
        %Reducimos paso de tiempo:
        Deltatn = Deltatn*max(0.3, (0.9*Tol/en)^(1/orden));
    else
        %Ajustamos paso de tiempo:
        Deltatn = Deltatn*min((0.9*Tol/en)^(1/orden), 2.0);
        %Aceptamos soluci'on:
        break
    end

    %Salir si el paso de tiempo es demasiado peque'no:
    if Deltatn<1e-12
        error('Se ha alcanzado el paso minimo de tiempo')
    end

end

%Guardamos soluci'on:
tv = cat(2, tv, tnp1);
ym = cat(2, ym, ynp1);

%Actualizamos variables:
tn = tnp1;
yn = ynp1;

end

%Funci'on para resolver etapa ii con Newton-Raphson. Se define aqu'i
%porque no se pueden definir funciones dentro de un bucle for o while.
%Esta funci'on tiene acceso a las variables t_ii, b_ii, etc. definidas
%arriba.

```

```
function [r,J] = ResiduoFun(y, CalcJ)

    %Evaluamos derivada y Jacobiano:
    [f,df_dy] = fun(t_ii, y, CalcJ);

    %Funci'on residuo y Jacobiano:
    r = y - aux_ii - Deltatn*A(ii,ii)*f;
    if CalcJ
        J = speye(length(y)) - Deltatn*A(ii,ii)*df_dy;
    else
        J = NaN;
    end

end

end
```

Published with MATLAB® R2025a

```

%Devuelve vector fila tv=[t_0, t_1, ..., t_n] y matriz
% ym = [y_0, y_1, ..., y_n], donde y_i = y(t_i) es un vector columna:
% fun es una función de la forma f = fun(t, y)
function [tv, ym] = RungeKuttaE(fun, t0, y0, metodo, Deltat, tf)

    %Condicional inicial:
    tn      = t0;
    yn      = y0(:);
    tv      = [tn];
    ym      = [yn];

    %Cargamos coeficientes:
    [A,b,c,s] = CoefsRK(metodo);

    %Matriz para guardar derivadas en las etapas intermedias:
    fm      = zeros(length(yn), s);

    %Bucle hasta llegar a tf:
    while tn < tf

        %Derivada en t^n:
        fm(:,1) = fun(tn, yn);

        %Paso de tiempo:
        tnp1 = tn + Deltat;
        if tnp1 > tf
            tnp1 = tf;
        end
        Deltatn = tnp1 - tn;

        %Recorremos etapas:
        for ii=2:s
            t_ii = tn + c(ii)*Deltatn;
            S = 0.0;
            for jj=1:ii-1
                S = S + A(ii,jj)*fm(:,jj);
            end
            y_ii = yn + Deltatn*S;
            fm(:,ii) = fun(t_ii, y_ii);
        end

        %Solución en t^(n+1):
        S = 0.0;
        for jj=1:s
            S = S + b(jj)*fm(:,jj);
        end
        ynp1 = yn + Deltatn*S;

        %Guardamos solución:
        tv = cat(2, tv, tnp1);
        ym = cat(2, ym, ynp1);
    end

```

```
    %Actualizamos variables:
    tn      = tnp1;
    yn      = ynp1;

end

end
```

Published with MATLAB® R2025a

```

%Devuelve vector fila tv=[t_0, t_1, ..., t_n] y matriz
% ym = [y_0, y_1, ..., y_n], donde y_i = y(t_i) es un vector columna:
function [tv, ym] = RungeKuttaI_Anderson1(fun, t0, y0, metodo, Deltat, tf)

    %Condici'on inicial:
    tn      = t0;
    yn      = y0(:);
    tv      = [tn];
    ym      = [yn];

    %Cargamos coeficientes:
    [A,b,c,s] = CoefsRK(metodo);

    %Matriz para guardar derivadas:
    fm       = zeros(length(yn), s);

    %Bucle hasta llegar a tf:
    while tn<tf

        %Derivada en t^n:
        [fm(:,1), df_dy_n] = fun(tn, yn, true);

        %Paso de tiempo:
        tnp1      = tn + Deltat;
        if tnp1>tf
            tnp1    = tf;
        end
        Deltatn    = tnp1-tn;

        %Matriz jacobiana R y factorizaci'on (suponemos matriz llena):
        Rm        = eye(length(yn)) - Deltatn*A(s,s)*df_dy_n;
        [L,U,P]    = lu(Rm);

        %Recorremos etapas:
        for ii=2:s
            %Variables auxiliares:
            t_ii      = tn + c(ii)*Deltatn;
            S          = 0.0;
            for jj=1:ii-1
                S      = S + fm(:,jj)*A(ii,jj);
            end
            aux_ii     = yn + Deltatn*S;
            %Resolver:
            [y_ii, ~, flag] = Anderson1(@gfun, yn, 1e-10, 100, 10);
            if flag<0
                warning('Anderson no ha convergido')
            end
            %Guardar derivada:
            fm(:,ii)    = fun(t_ii, y_ii, false);
        end

        %Soluci'on en t^(n+1):

```

```

S          = 0.0;
for jj=1:s
    S      = S + fm(:,jj)*b(jj);
end
ynp1      = yn + Deltatn*S;

%Guardamos soluci'on:
tv        = cat(2, tv, tnp1);
ym        = cat(2, ym, ynp1);

%Actualizamos variables:
tn        = tnp1;
yn        = ynp1;

end

%Funci'on para resolver etapa ii con Newton-Raphson. Se define aqu'i
%porque no se pueden definir funciones dentro de un bucle for o while.
%Esta funci'on tiene acceso a las variables t_ii, b_ii, etc. definidas
%arriba.
function g = gfun(y)

    %Evaluamos f:
    [f,~]   = fun(t_ii, y, false);

    %Residuo:
    r       = y - aux_ii - Deltatn*A(ii,ii)*f;

    %Residuo preconditionado (suponemos matriz llena):
    g       = U\ (L\ (P*r));

end

end

```

Published with MATLAB® R2025a

```

%Devuelve vector fila tv=[t_0, t_1, ..., t_n] y matriz
% ym = [y_0, y_1, ..., y_n], donde y_i = y(t_i) es un vector columna:
function [tv, ym] = RungeKuttaI(fun, t0, y0, metodo, Deltat, tf)

    %Condici'on inicial:
    tn      = t0;
    yn      = y0(:);
    tv      = [tn];
    ym      = [yn];

    %Cargamos coeficientes:
    [A,b,c,s] = CoefsRK(metodo);

    %Matriz para guardar derivadas:
    fm       = zeros(length(yn), s);

    %Bucle hasta llegar a tf:
    while tn<tf

        %Derivada en t^n:
        fm(:,1) = fun(tn, yn, false);

        %Paso de tiempo:
        tnp1     = tn + Deltat;
        if tnp1>tf
            tnp1 = tf;
        end
        Deltatn  = tnp1-tn;

        %Recorremos etapas:
        for ii=2:s
            %Variables auxiliares:
            t_ii      = tn + c(ii)*Deltatn;
            S          = 0.0;
            for jj=1:ii-1
                S      = S + fm(:,jj)*A(ii,jj);
            end
            aux_ii     = yn + Deltatn*S;
            %Resolver:
            [y_ii, nIters, flag]= NewtonRaphson1(@ResiduoFun, yn, 1e-10, 100);
            if flag<0
                warning('NewtonRaphson no ha convergido')
            end
            %Guardar derivada:
            fm(:,ii)   = fun(t_ii, y_ii, false);
        end

        %Soluci'on en t^(n+1):
        S            = 0.0;
        for jj=1:s
            S        = S + fm(:,jj)*b(jj);
        end
    end

```

```

    ynp1      = yn + Deltatn*S;

    %Guardamos soluci'on:
    tv        = cat(2, tv, tnp1);
    ym        = cat(2, ym, ynp1);

    %Actualizamos variables:
    tn        = tnp1;
    yn        = ynp1;

end

%Funci'on para resolver etapa ii con Newton-Raphson. Se define aqu'i
%porque no se pueden definir funciones dentro de un bucle for o while.
%Esta funci'on tiene acceso a las variables t_ii, b_ii, etc. definidas
%arriba.
function [r,J] = ResiduoFun(y, CalcJ)

    %Evaluamos derivada y Jacobiano:
    [f,df_dy] = fun(t_ii, y, CalcJ);

    %Funci'on residuo y Jacobiano:
    r          = y - aux_ii - Deltatn*A(ii,ii)*f;
    if CalcJ
        J      = speye(length(y)) - Deltatn*A(ii,ii)*df_dy;
    else
        J      = NaN;
    end

end

end
end

```

Published with MATLAB® R2025a